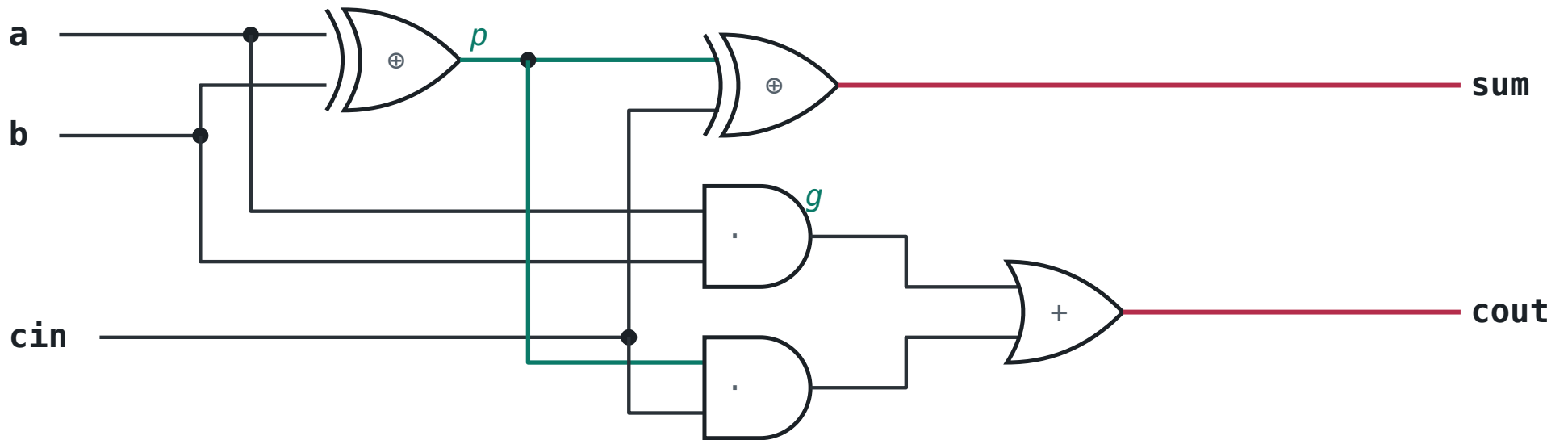


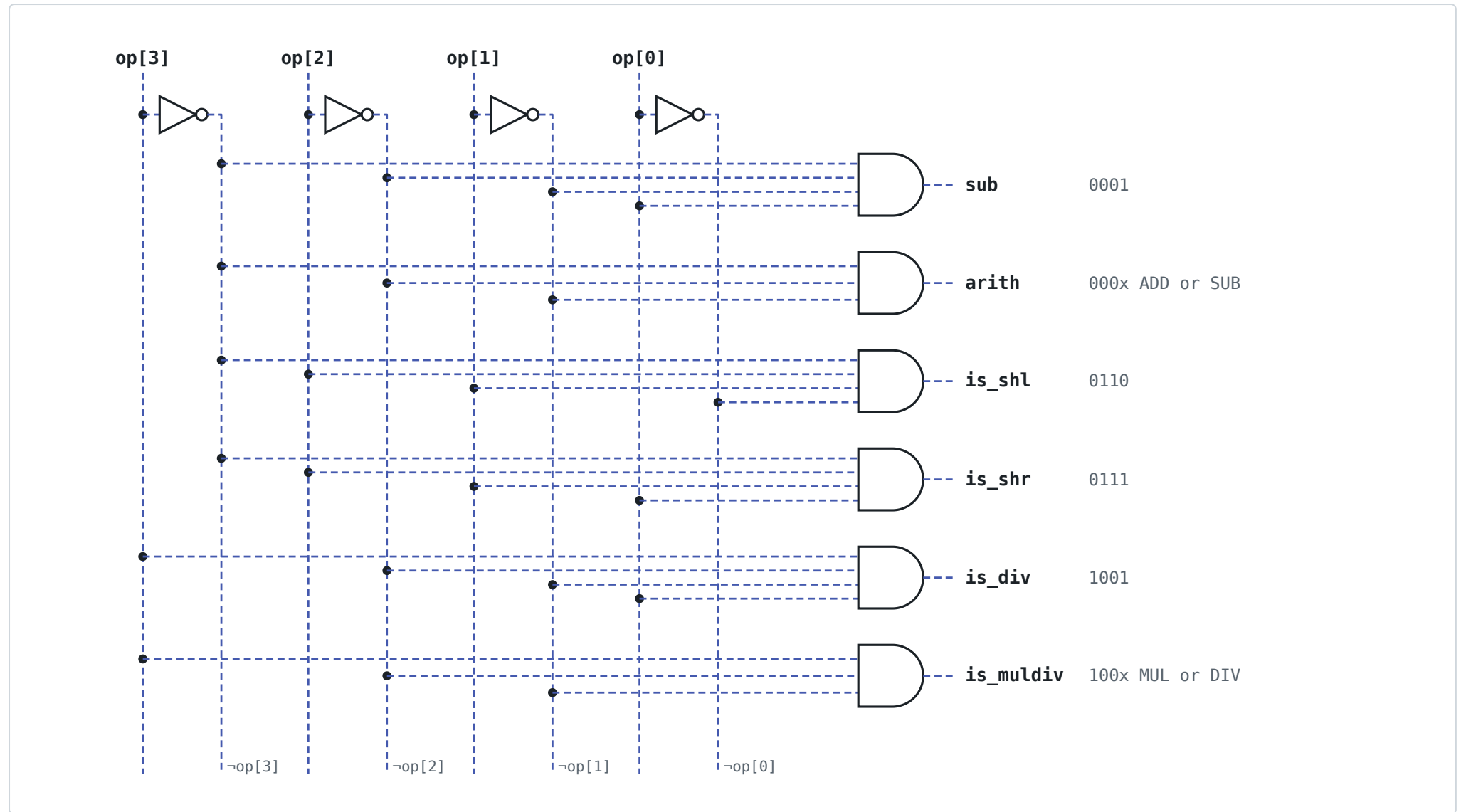
1 · Full adder (full_adder.v)

$\text{sum} = a \oplus b \oplus \text{cin}$, $\text{cout} = a \cdot b + (a \oplus b) \cdot \text{cin}$. 2 XOR, 2 AND, 1 OR. Used 4 times in adder_subtractor_4bit.



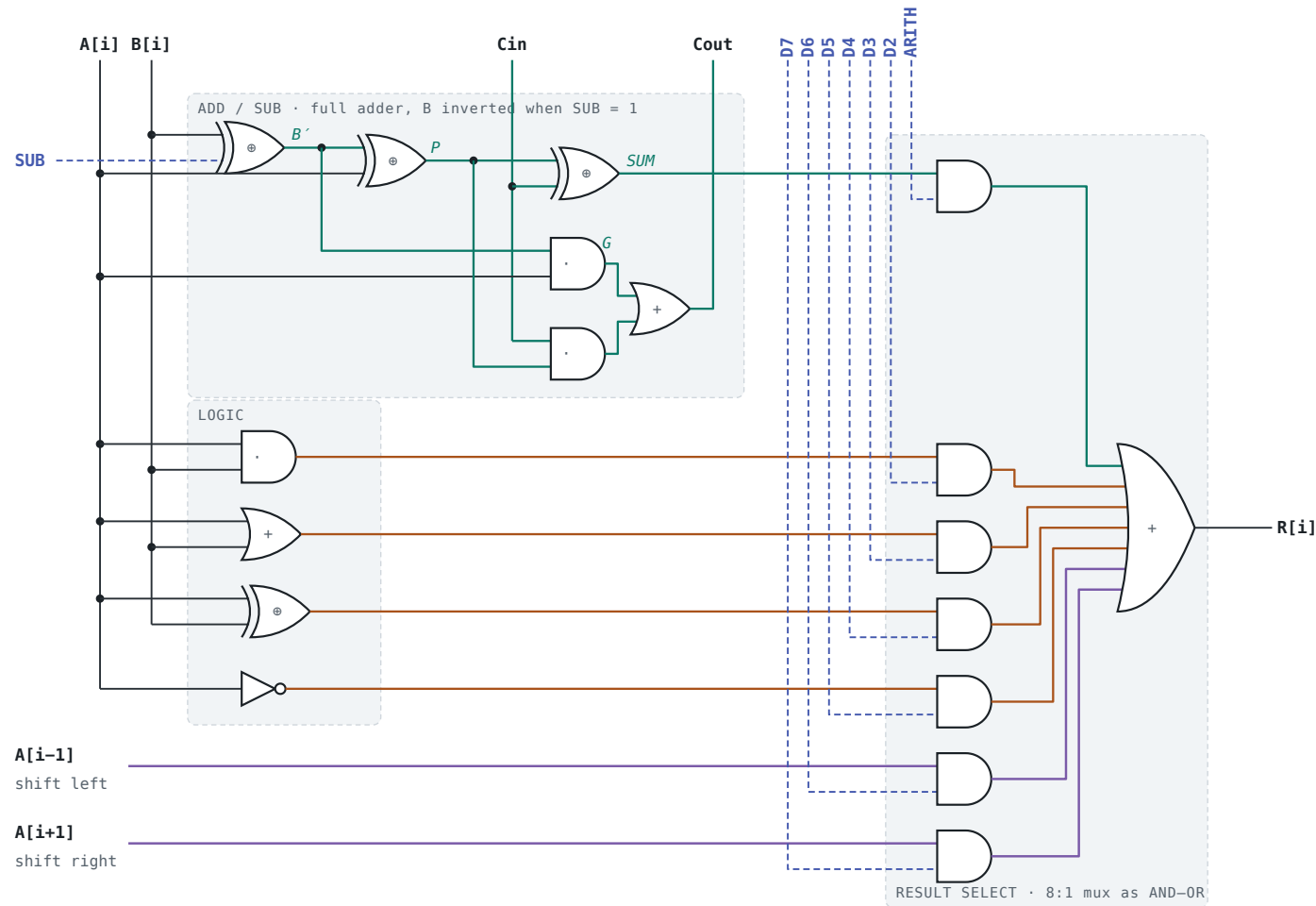
2 · Opcode decoder (control_unit.v)

4 NOT + 6 AND gates turn opcode[3:0] into the control signals. arith (000x) and is_muldiv (100x) ignore bit 0.



3 · One bit slice of the ALU datapath (repeated ×4)

Adder/subtractor bit ($B \oplus \text{SUB}$ then full adder), logic gates, shift wiring, and the result select as AND-OR. ARITH = opcode 000x; D2-D7 = opcodes 0010 AND, 0011 OR, 0100 XOR, 0101 NOT, 0110 SHL, 0111 SHR.

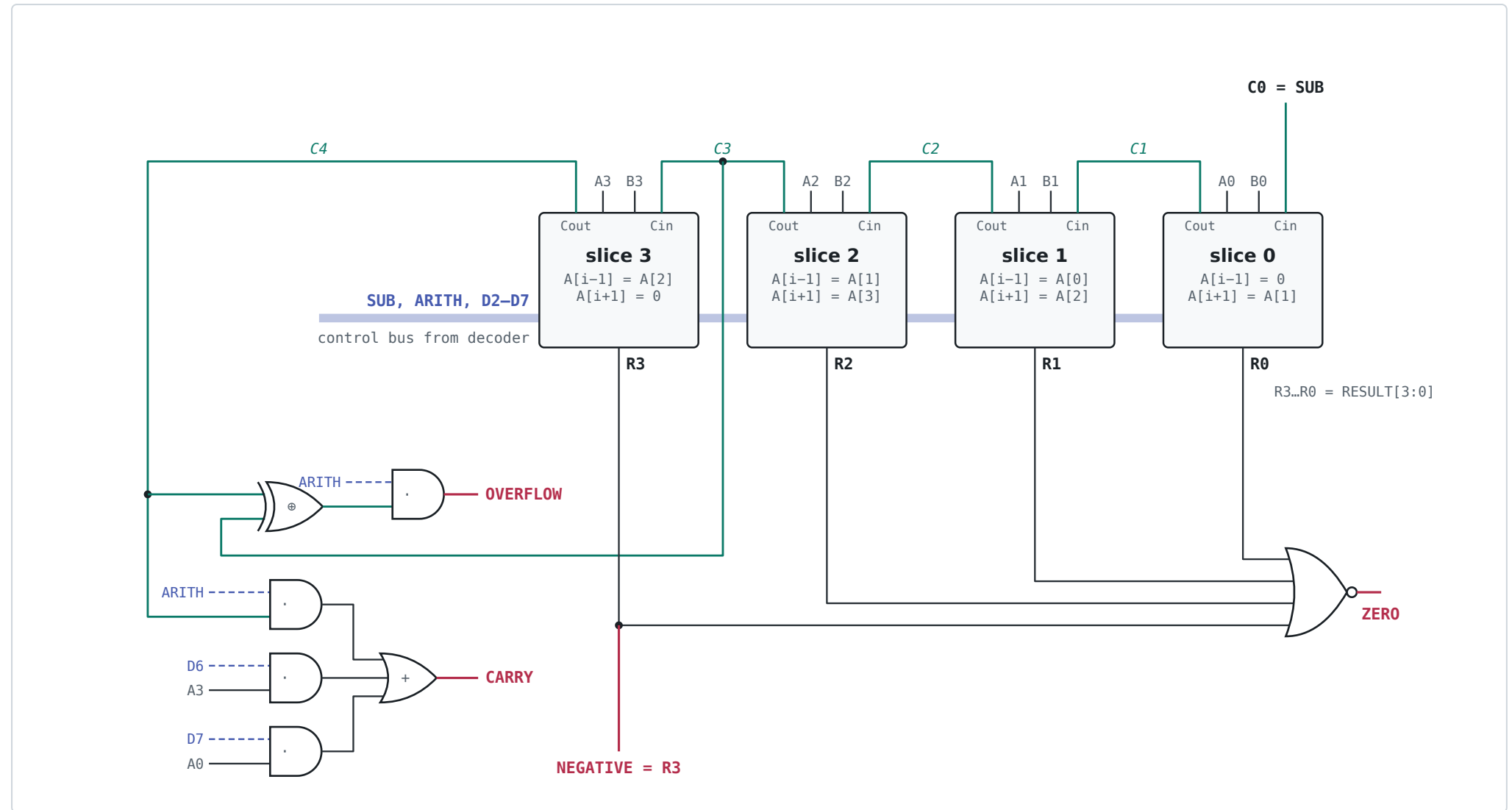


— adder & carry — logic results — shift wiring --- control / select — outputs & flags

● junction – crossing lines without a dot are not connected

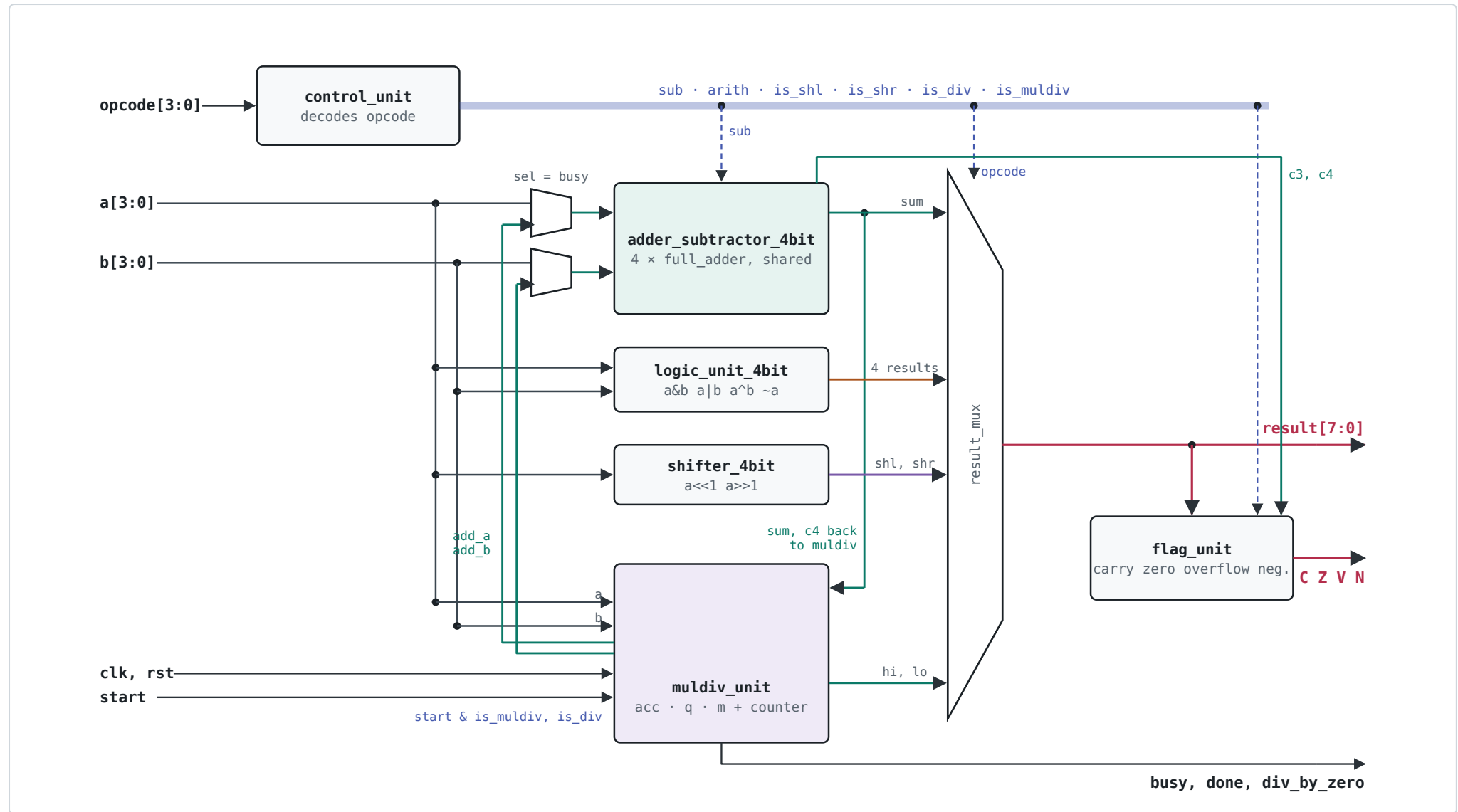
4 · Four slices, ripple carry and flag logic

Carry ripples slice 0 → slice 3 ($C0 = \text{SUB}$). $\text{OVERFLOW} = \text{ARITH} \cdot (C3 \oplus C4)$; $\text{CARRY} = \text{ARITH} \cdot C4 + D6 \cdot A3 + D7 \cdot A0$; $\text{ZERO} = \text{NOR}$ of the result; $\text{NEGATIVE} = R3$. (For MUL/DIV, ZERO checks all 8 result bits and NEGATIVE is 0.)



5 · Complete ALU block diagram (alu_4bit.v)

How the files connect. MUL/DIV (muldiv_unit) uses flip-flops and borrows the shared adder while busy = 1.



— adder & carry — logic results — shift wiring --- control / select — outputs & flags

● junction – crossing lines without a dot are not connected